

02a — Git

Key Concepts

Term	Definition
Git	Distributed version control system — every developer has full repo history locally
Repository	Project folder + full change history stored in <code>.git</code>
Commit	Snapshot of files at a point in time, with hash, author, message
Branch	Isolated line of development — changes don't affect other branches
HEAD	Pointer to the current commit on the current branch
Merge	Combines one branch's changes into another
Rebase	Replays commits on top of another branch — creates linear history
Stash	Temporarily shelves uncommitted changes

The Three Areas of Git



Area	Purpose
Working Directory	Files on disk you can see and edit
Staging Area	Pre-commit holding area — choose exactly what goes into next commit
Repository	Permanent history — all commits stored in <code>.git</code>

Why staging exists: Lets you group related changes into meaningful commits even when many files changed.

Essential Commands

Command	What it does
<code>git init</code>	Create new repo
<code>git clone <url></code>	Copy remote repo locally
<code>git status</code>	Show changed/staged/untracked files
<code>git add <file></code>	Stage a file

Command	What it does
<code>git commit -m "msg"</code>	Save staged changes as commit
<code>git log</code>	View commit history
<code>git branch <name></code>	Create a branch
<code>git checkout <branch></code>	Switch branch
<code>git merge <branch></code>	Merge branch into current
<code>git pull</code>	Fetch + merge from remote
<code>git push</code>	Send commits to remote
<code>git fetch</code>	Download remote changes without merging
<code>git stash</code>	Shelve uncommitted changes
<code>git diff</code>	Show unstaged changes

Branching Strategies

Strategy	How it works	Used by
Feature Branch + PR	Each feature on its own branch, merged via PR with review	Most companies
Git Flow	<code>main</code> , <code>develop</code> , <code>feature/*</code> , <code>release/*</code> , <code>hotfix/*</code>	Scheduled-release products
Trunk-Based Development	Everyone commits to <code>main</code> , feature flags hide unfinished work	Google, Netflix, Facebook

Merge vs Rebase

	Merge	Rebase
Result	Merge commit — preserves branch history	Linear history — replays commits
Safe on shared branches?	Yes	No — rewrites history
Use case	PRs, team branches	Local cleanup before pushing

Interview Questions

Q: What is the difference between Git and GitHub? A: Git is the version control tool. GitHub is a cloud platform that hosts Git repos and adds collaboration features (PRs, CI/CD, access control).

Q: Explain the three areas of Git. A: Working directory (files on disk), staging area (what's queued for the next commit), repository (permanent history in `.git`).

Q: What is the difference between `git pull` and `git fetch`? A: `git fetch` downloads changes from remote but doesn't apply them. `git pull` fetches and immediately merges into your current branch.

Q: When would you use rebase instead of merge? A: Rebase locally before pushing to clean up messy commit history into a linear sequence. Never rebase shared/public branches — it rewrites history and breaks other developers' refs.